

Q1. Determine the time complexity:

```
For(int i=0;i<n;i++)
```

```
{
```

```
cout<<l;
```

```
}
```

Answer: liner time – $O(n)$;

Q2. Determine the time complexity:

```
for(int i=0;i<n;i++)
```

```
{
```

```
    for(int j=0;j<n;j++)
```

```
    {
```

```
        cout<<"*";
```

```
    }
```

```
}
```

Answer: quadratic time = $O(n^2)$;

Q3.Determine the time complexity:

```
for(int i=0;i<n;i++)
```

```
{
```

```
    for(int j=0;j<i;j++)
```

```
    {
```

```
        cout<<"*";
```

```
    }
```

```
}
```

Answer: Quadratic time – $O(n^2)$.

Q4. Determine the time complexity:

```
for(int i=1;i<n;i*=2)
{
    cout<<i;
}
```

Answer: $O(\log n)$;

Q5. Determine the time complexity:

```
for(int i=n;i>1;i/=2)
{
    cout<<i;
}
```

Answer: $O(\log n)$;

Q6. Determine the time complexity:

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=log(n);j++)
    {
        cout<<"*";
    }
}
```

Answer: $O(n \log n)$;

Q7. Determine the time complexity:

```
while(n>0)
```

```
{
```

```
    n--;
```

```
}
```

Answer: $O(n)$;

Q8. Determine the time complexity:

```
while(n>1)
```

```
{
```

```
    n/=2;
```

```
}
```

Answer: $O(\log n)$;

Q9. Determine the time complexity:

```
for(int i=1;i<=n;i++)
```

```
{ }
```

```
for(int j=1;j<=n;j++)
```

```
{ }
```

Answer: $O(n)$;

Q10. Determine the time complexity:

```
for(int i=1;i<=n;i++)
```

```
{
```

```
}
```

```
for(int j=1;j<=n*n;j++)
```

```
{
```

```
}
```

Answer: $O(n^2)$;

Q11. Determine the time complexity:

```
for(int i=1;i<=n;i++)  
{  
    for(int j=1;j<=n*n;j++)  
    {  
    }  
}
```

Answer: $O(n^2)$;

Q12. Determine the time complexity:

```
for(int i=1;i<=n;i*=3)  
{  
}
```

Answer: $O(\log_3 n)$;

Q13. Determine the time complexity:

```
for(int i=1;i<=n;i++)  
{  
    for(int j=1;j<=i*i;j++)  
    {  
    }  
}
```

Answer: $O(n^3)$;

Q14. Determine the time complexity:

```
void fun(int n)  
{  
    if(n==0)  
        return;  
    fun(n-1);  
}
```

```
}
```

Answer: $O(n)$;

Q15. Determine the time complexity:

```
void fun(int n)
```

```
{
```

```
    if(n<=1)
```

```
        return;
```

```
    fun(n/2);
```

```
}
```

Answer: $O(\log n)$;

Q16. Determine the time complexity:

```
void fun(int n)
```

```
{
```

```
    if(n==0)
```

```
        return;
```

```
    fun(n-1);
```

```
    fun(n-1);
```

```
}
```

Answer: **$O(2^n)$**

Q17. Determine the time complexity:

```
int fib(int n)
```

```
{
```

```
    if(n<=1)
```

```
        return n;
```

```
    return fib(n-1)+fib(n-2);
```

```
}
```

Answer: **$O(2^n)$**

Q18. Determine the time complexity:

```
int fact(int n)
{
    if(n<=1)
        return 1;
    return n*fact(n-1);
}
```

Answer: $O(n)$

Q19. Determine the time complexity:

```
void fun(int n)
{
    if(n<=0)
        return;
    fun(n-1);
    cout<<n;
}
```

Answer: $O(n)$

Q20. Determine the time complexity:

```
void fun(int n)
{
    if(n<=1)
        return;
    fun(n/2);
    cout<<n;
}
```

Answer: $O(\log n)$

Q21. Solve the recurrence relation:

$$T(n) = T(n-1) + 1$$

Answer: $T(n) = O(n)$

Q22. Solve the recurrence relation:

$$T(n) = T(n-1) + n$$

Answer: $T(n) = O(n^2)$

Q23. Solve the recurrence relation:

$$T(n) = T(n/2) + 1$$

Answer: $T(n) = O(\log n)$

Q24. Solve the recurrence relation:

$$T(n) = 2T(n-1) + 1$$

Answer: $T(n) = O(2^n)$ – exponential time;

Q25. Solve the recurrence relation:

$$T(n) = T(n/2) + n$$

Answer: $T(n) = O(n)$;

Bonus GATE-Level Questions

Q26.

```
.for(int i=1;i<=n;i++)
```

```
{
```

```
    for(int j=1;j<=i;j*=2)
```

```
    {
```

```
    }
```

```
}
```

Answer: $O(n \log n)$;

Q27. Determine the complexity.

```
for(int i=1;i<=n;i*=2)
{
    for(int j=1;j<=n;j++)
    {
    }
}
```

Answer: $O(n \log n)$;

Q28. Determine the complexity.

```
for(int i=n;i>=1;i--)
{
    for(int j=i;j>=1;j--)
    {
    }
}
```

Answer: Quadratic time – $O(n^2)$;

Q29. Determine the complexity.

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=n;j*=2)
    {
    }
}
```

Answer: $O(n \log n)$;

Q 30. Determine the complexity.

```
void fun(int n)
```

```
{
```

```
    if(n<=1)
```

```
        return;
```

```
    fun(n/2);
```

```
    fun(n/2);
```

```
}
```

Answer: linear time – $O(n)$;